AI ENGINEERING GUIDE

Token Optimization Techniques

Reduce Cost. Improve Latency. Scale AI Systems.

A practical guide for AI Engineers building production AI applications.

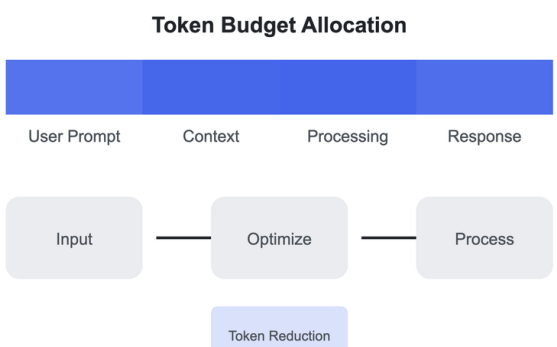
Swipe to learn all 10 techniques →


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

OVERVIEW

The Production AI Engineer's Token Optimization Toolkit

01 **Prompt Compression** Remove unnecessary verbosity	**02** **Context Filtering** Send only relevant info
03 **Dynamic Context Injection** Inject only when required	**04** **Semantic Caching** Reuse previous responses
05 **Conversation Summarization** Compress chat history	**06** **Context Minification** Reduce document size
07 **RAG Optimization** Fewer but better chunks	**08** **Structured Outputs** JSON instead of long text
09 **Model Routing** Smallest capable model	**10** **Token-Aware Agent Design** Prevent context waste




AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

TECHNIQUE 01

Prompt Compression

Many prompts contain unnecessary words that inflate token count without improving output quality.

BAD

"Please carefully analyze the following information and provide a detailed response."

BETTER

"Analyze and summarize."

OPTIMIZATION IDEAS

- Remove repeated instructions
- Shorten system prompts
- Eliminate unnecessary examples
- Use reusable prompt templates

20–50% token reduction in prompts with tighter instructions.


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

TECHNIQUE 02

Context Filtering

Most systems send too much information. Only a fraction is actually useful.

10
RETRIEVED CHUNKS

→

3
ACTUALLY USEFUL

FILTER CONTEXT USING

- ✓ Similarity score thresholds
- ✓ Metadata filters
- ✓ Access permissions
- ✓ Re-ranking models

↓ **Less context. Better answers. Lower cost.**


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

TECHNIQUE 03

Dynamic Context Injection

Many systems always send everything regardless of what the user actually needs.

ALWAYS INJECTED (PROBLEM)

- ✕ User Profile
- ✕ Policies
- ✕ Documentation
- ✕ Billing Docs
- ✕ Product Data
- ✕ Chat History

INJECT ONLY WHAT FITS THE QUERY

User asks: **"Reset my password."**

✓ Auth docs ✕ Billing docs ✕ Pricing docs

Significant token savings on every single request.


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

TECHNIQUE 04

Semantic Caching

The same questions appear repeatedly in different phrasings. Most systems process each one from scratch.

REPEATED QUERIES (PROBLEM)

- "Reset my password"
- "Change password"
- "Forgot password"

CACHE SOLUTION

Query Embedding → Cache Store → Reuse Response

BENEFITS

- ✓ Faster responses on repeated queries
- ✓ Lower API cost per request
- ✓ Reduced model usage overall


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

TECHNIQUE 05

Conversation Summarization

Long conversations consume large context windows and drive up cost with every turn.

BEFORE

100

Messages

~8,000

Tokens per call

AFTER

Summary

~500

Recent Messages

~300

Total tokens

HOW IT WORKS

- Summarize older turns into a compact block
- Keep only the most recent N messages verbatim
- Prepend summary at the start of each new call

Keep context quality while dramatically reducing token usage.



AI Coach John Follow

PROITBRIDGE Strive For Better Future

Repost

TECHNIQUE 06

Context Minification

Retrieved documents are often much larger than necessary. Most of the content is irrelevant to the query.

EXTRACT ONLY WHAT MATTERS

- ✓ Relevant paragraphs
- ✓ Key facts and figures
- ✓ Important sections

BEFORE	→	AFTER
5,000 token document		700 token summary

Smaller prompts. Faster generation. Lower cost per call.


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

TECHNIQUE 07

RAG Optimization

More chunks does not mean better answers. Poor retrieval inflates tokens and degrades quality.

BAD RAG

20
chunks retrieved

8,000
tokens

GOOD RAG

3
highly relevant chunks

1,200
tokens

OPTIMIZATION TECHNIQUES

- Better chunking strategies
- Re-ranking retrieved results
- Hybrid search (dense + sparse)
- Metadata filters before retrieval

Higher retrieval quality with significantly fewer tokens.


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

</> TECHNIQUE 08

Structured Outputs

LLMs often generate unnecessary explanations and preamble. Force structured output to eliminate token waste.

→ **Instead, generate:**

```
{
  "priority": "high",
  "status": "approved",
  "risk": "low"
}
```

BENEFITS

- ✓ Smaller output responses
- ✓ Easier downstream automation
- ✓ Better reliability and parsing

↓ **Less output token consumption on every generation call.**


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

TECHNIQUE 09

Model Routing

Not every task needs the largest model. Routing to smaller models saves significant cost.



Spam Detection

↓

Small Model



Complex Planning

↓

Large Model

ROUTING FLOW

User Query → Classifier → Best Model

$ Lower cost while maintaining the quality users expect.

TECHNIQUE 10

Token-Aware Agent Design

Agents often waste tokens through poor memory and planning patterns.

COMMON ISSUES

- ✕ Re-reading documents each step
- ✕ Repeating tool outputs in prompt
- ✕ Sending full memory every step
- ✕ Duplicate reasoning loops

SOLUTIONS

- ✓ Memory compression
- ✓ Selective retrieval
- ✓ Tool output summarization
- ✓ Smart step planning

More efficient agent workflows with lower cost per task.


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

KEY TAKEAWAY

The Best Token Is The One You Never Send

HIGHEST ROI TECHNIQUES

#1 Context Filtering

#2 Dynamic Context Injection

#3 Conversation Summarization

#4 Semantic Caching

#5 RAG Optimization

Production AI systems commonly reduce token usage by **30–70%** using these techniques.


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost